

The Multilevel USE Extensions

MLM-USE / CatUSE Project

August 29, 2026

Contents

I	Getting Started	1
1	Introduction	3
1.1	What Is USE?	3
1.2	Three Layers: USE, MLM-USE, CatUSE	3
1.3	Terminology at a Glance	4
1.4	How to Read This Manual	4
2	Installing and Building	7
2.1	Prerequisites	7
2.2	Getting the Source	7
2.3	Module Layout	7
2.4	Building with Maven	8
2.5	Build Artifacts	8
3	Running the System	9
3.1	Running the Assembled Distribution	9
3.2	Running Directly with <code>java -jar</code>	9
3.3	Double-Clicking the Jar	9
3.4	Command-Line Options	9
3.5	The Shell Runs Underneath the GUI, Too	10
3.6	Opening a Specification: Four Entry Points	10
3.7	Loading a Specification from the Command Line	11
3.8	A Quick Sanity Check	11
4	A Quickstart Tutorial	13
II	Modeling in Plain USE	15
5	Class Diagrams	17
6	OCL Constraints	19
7	The OCL Standard Library	21
8	System States and the Shell	23
9	Validating Specifications	25
10	The Graphical User Interface	27
11	Model Unit Tests	29

III	Multi-Level Modeling with MLM-USE	31
12	Concepts	33
13	MLM-USE Syntax in Detail	35
14	Multi-Level Shell Commands	37
15	Worked Examples	39
16	Validating Multi-Level Models	41
16.1	What “Validating a Model” Means in USE	41
16.2	The Multi-Level Blind Spot	42
16.3	Two Different Ways to Think About the Problem	42
17	The Flattening Bridge	45
17.1	The Idea	45
17.2	FlatUseSystemFactory	45
17.3	What’s Verified	46
17.4	The Bundled Model-Finder Plugin	46
17.5	What’s Not Yet Wired Up	46
IV	CatUSE and CatMLM	49
18	Motivation	51
19	CatMLM Syntax in Detail	53
20	Known Expressiveness Gaps	55
20.1	Association classes are per-model, not multi-level	55
20.1.1	The restriction	55
20.1.2	Why	55
20.1.3	What to write instead	55
20.2	CatMLM’s <code>instanceOf</code> only ever targets a level’s declared parent	56
20.2.1	The restriction	56
20.2.2	Why this isn’t a gap in practice	56
20.2.3	What to write instead	56
20.3	Other CatMLM gaps (unchanged from earlier passes)	56
21	Worked Examples	57
22	The CatUSE-to- $\text{\LaTeX}$ Study Tool	59
V	Extending the Implementation	61
23	Codebase Tour	63
24	The Grammar and Parser	65
25	AST and Compilation Pipeline	67
26	The Core M-Model	69

<i>CONTENTS</i>	5
27 The Multi-Model and Multi-Level-Model Classes	71
28 Flattening and Rendering	73
29 Testing the Compiler	75
30 Releasing a New Version and Writing Plugins	77
 VI History	 79
31 History of USE	81
32 History of the MLM-USE/CatUSE Fork	83
 VII Appendices	 85
A The USE Grammar	87
B The MLM-USE Grammar	99
C The CatMLM Grammar	103
D Keyword Reference	109
E Shell Command Reference	111

Part I

Getting Started

Chapter 1

Introduction

This manual documents a research fork of USE that adds **multi-level modeling** (MLM-USE) and a terser front-end syntax for it (CatUSE, also called CatMLM). It is written for someone who wants to actually write, check, and reason about models in this system — not just read about the theory behind it.

1.1 What Is USE?

USE (the “UML-based Specification Environment”) is a tool for specifying and validating models written in a subset of UML together with the Object Constraint Language (OCL). A USE specification is a plain text file describing classes, associations, and OCL invariants; USE compiles it, lets you populate a running *system state* with objects and links (by hand, via scripts, or interactively), and continuously checks that state against the model’s constraints. Originally developed at the University of Bremen, it is both a teaching tool for UML/OCL and a research platform – this fork is an example of the latter.

Everything in Part II describes plain USE as it stands in this fork today, independent of anything multi-level.

1.2 Three Layers: USE, MLM-USE, CatUSE

This fork (developed at Ben-Gurion University, building on the “Mediation-Based MLM in USE” line of work) adds two more layers on top of plain USE, each strictly **additive**: every plain USE model is already a valid input to the layers above it.

Layer	Adds	Entry keyword
USE	Classes, associations, OCL constraints, SOIL	model
MLM-USE	Levels, clabstracts, mediators, powerclasses	MLM
CatUSE	Terser syntax that desugars into MLM-USE’s AST	category

MLM-USE (Part III) is where multi-level modeling actually lives: a class can simultaneously be an ordinary class **and** an instance of another class one level up (a *clabstract*), letting a single model describe several ontological levels – for example, a product catalog level whose classes are themselves instances of a “kind of product” level – without collapsing them into one flat model or duplicating information across separate models.

CatUSE (Part IV, also called CatMLM in the source) is a front-end grammar layered on top of MLM-USE. It does not add any new semantics of its own: a CatUSE specification is parsed and then desugared into exactly the same abstract syntax tree that an equivalent, more verbose MLM-USE specification would produce. Compare:

```
category Car < Vehicle : Product
  attributes
    price : Integer
end
```

Listing 1.1: CatUSE (terse)

against the MLM-USE it desugars into (a separate `clabject` statement naming the same cross-level relationship):

```
class Car < Vehicle
  attributes
    price : Integer
end

clabject Car : Product end
```

Listing 1.2: The MLM-USE it desugars to

CatUSE currently covers less ground than MLM-USE – some things (see Chapter 20) can only be written in full MLM-USE. Which one to reach for is mostly a matter of how much of a model is genuinely multi-level: CatUSE reads better when most classes in a level participate in cross-level instantiation; plain MLM-USE keeps more control when only a few do, or when a construct CatUSE cannot express is needed.

1.3 Terminology at a Glance

A handful of terms recur throughout this manual. This is only a pointer to where each is properly introduced, not a substitute for reading that section.

Model A named collection of classes, associations, and constraints – the unit USE compiles and checks. See Chapter 5.

Level One “floor” of a multi-level model. See Chapter 12.

Clabject A class that is also, simultaneously, an instance of a class at a higher level. The name blends “class” and “object”. See Chapter 12.

Powerclass The higher-level class a clabject is an instance of. See Chapter 12.

Mediator A declaration that relates classes across two adjacent levels. See Chapter 12.

Category CatUSE’s keyword for a classifier declaration that may also carry cross-level instantiation directly on its own header. See Chapter 19.

1.4 How to Read This Manual

Part I gets the system installed, built, and running, and ends with a short guided walkthrough (Chapter 4). Part II documents plain USE in full. Parts III and IV cover MLM-USE and CatUSE respectively, each in the same shape: concepts, syntax, worked examples. Part V is for anyone modifying this fork’s own implementation, not just writing models in it. Part VI covers where USE and this fork came from. The appendices are lookup references (grammars, keywords, shell commands), not narrative – go there when you need to check a detail, not to read start to end.

Code listings are syntax-highlighted: hard (fully reserved) keywords in one color, soft (contextual) keywords in another – see Chapter [D](#) for what that distinction means and why it matters. Boxed notes call out things worth pausing on:

Note

This manual assumes no prior familiarity with USE. If you already know plain USE and only want the multi-level material, Part [I](#) can be skimmed and Part [II](#) skipped entirely.

Chapter 2

Installing and Building

2.1 Prerequisites

You need:

- A JDK for Java 21 or later (the build sets both `maven.compiler.source` and `maven.compiler.target` to 21; older JDKs cannot compile this codebase).
- Apache Maven (any reasonably current 3.x release).

No other tools are required to build and run the system itself. Building this manual is a separate matter, covered by its own `Makefile` in `new-documentation/`, not by the project's Maven build.

2.2 Getting the Source

```
git clone https://github.com/sturmarn/useBGU.git
cd useBGU
git checkout MLM-USE
```

All multi-level and CatUSE work described in this manual lives on the `MLM-USE` branch of this fork, not on upstream `USE`.

2.3 Module Layout

The build is a single Maven reactor with three modules:

Module	Contains
<code>use-core</code>	Grammars, parser, AST, the M-model (classes/associations/etc.), the compiler and system-state engine
<code>use-gui</code>	The Swing GUI, the interactive shell, and the <code>Main</code> entry point
<code>use-assembly</code>	Packages the other two, plus scripts and bundled examples, into a runnable distribution

Chapters 23 onward go into this layout in much more detail; this chapter only covers enough to build and run the system.

2.4 Building with Maven

From the repository root:

```
mvn clean package
```

Maven’s reactor builds `use-core`, then `use-gui`, then `use-assembly`, in that order, in a single invocation – there is no separate per-module build step to run by hand.

Note

`mvn package` runs the unit test suite (via Surefire) as part of the build; a failing test fails the build. To also run the slower integration tests (Failsafe, classes named `*IT`, e.g. GUI smoke tests), use `mvn verify` instead – this is what CI runs.

2.5 Build Artifacts

After a successful build:

- `use-core/target/*.jar` – the compiler/engine as a library.
- `use-gui/target/use-gui.jar` – a single runnable jar with all dependencies bundled and a manifest `Main-Class` of `org.tzi.use.main.Main`. This is the jar Chapter 3 runs directly.
- `use-assembly/target/use-7.1.1.tar.gz` and `.zip` – a full distributable archive: a `bin/` launcher script, `lib/use-gui.jar`, bundled OCL extensions, plugin jars, and the example models shipped with the project.

The version number in the assembly’s filename tracks the project’s `pom.xml` version (7.1.1 at the time of writing); it will differ if that version has moved on since.

Chapter 3

Running the System

3.1 Running the Assembled Distribution

Unpack `use-assembly/target/use-7.1.1.tar.gz` (or the `.zip`) and run its launcher script:

```
tar xzf use-7.1.1.tar.gz
cd use-7.1.1
bin/use
```

The script resolves its own location, points at `lib/use-gui.jar` next to it, and forwards any arguments you give it verbatim to that jar (`bin/use [options] [spec_file] [cmd_file]`, covered below).

3.2 Running Directly with `java -jar`

Without unpacking a distribution, the freshly built jar runs on its own:

```
java -jar use-gui/target/use-gui.jar
```

This is the same jar `bin/use` wraps, so every option below behaves identically either way.

3.3 Double-Clicking the Jar

`use-gui.jar` carries a manifest `Main-Class`, so on a system where `.jar` files are associated with a Java launcher (typically the case on Windows and macOS out of the box; Linux desktop environments vary and often need that association set up manually) double-clicking it launches the GUI directly, with no arguments – equivalent to running `java -jar use-gui.jar` with no `spec_file`. Passing command-line options or a spec file to open on startup requires an actual command-line invocation; double-clicking cannot supply either.

3.4 Command-Line Options

The full usage message (`java -jar use-gui.jar -h`):

```
usage: use [options] [spec_file] [cmd_file]
```

options:

```
-c          compile only
-cp         compile and print specification
-disableCollectShorthand
```

```

        flag use of OCL shorthand notation as error
-oclAnyCollectionsChecks:W
        (W)arn|(E)rror|(I)gnore
-extendedTypeSystemChecks:W
        (W)arn|(E)rror|(I)gnore
-implicitTypes    Implicit variable typing in operations
-nogui           do not use GUI
-noplugins       do not use plugins
-h              print help
-H=path         home of use installation
-nr             suppress warnings about missing readline library
-q             reads spec_file, executes cmd_file, and checks constraints
               exit code is 1 if constraints fail, otherwise 0
-qv            like -q but with verbose output of constraint check
-v             print verbose messages
-vt            print verbose messages with time info
-V             print version info and exit

spec_file:      file containing a USE specification
cmd_file:      script file containing commands (will be executed on startup)

diagnostics:
  -debug       print lots of messages
  -p           print stack traces on errors
  -t           testing mode (less user-centric output)

```

The ones you will reach for most as a new user are `-nogui` (run headless), `-v` (see what’s happening during a slow compile), `-q` / `-qv` (batch-check a model and get a process exit code – useful for scripting or CI), and `-h` / `-V`. The `diagnostics` group and `-disableCollectShorthand`, `-oclAnyCollectionsChecks`, and `-extendedTypeSystemChecks` are aimed more at people working on the compiler itself (Part V) than at someone writing models.

3.5 The Shell Runs Underneath the GUI, Too

`-nogui` does not disable the interactive shell – it only suppresses the window. Looking at how startup actually proceeds: if a `spec_file` is given, it is compiled and a system state is created **before** the GUI or shell start; then, unconditionally, a shell thread is started; the GUI window is created only if `-nogui` was not passed. The GUI is a window wrapped around the same underlying engine the shell drives – not a separate mode. Chapter 8 covers the shell’s own command language in full.

3.6 Opening a Specification: Four Entry Points

The GUI’s File menu offers four distinct “Open...” actions, and they are **not** content-sniffed: each parses its input with a different grammar entry point, and you must pick the one matching what you actually wrote.

Action	Parses
Open specification...	A plain, single <code>model</code> (Part II)
Open multi specification...	Several ordinary models glued by <code>inter-*</code> sections, no levels
Open multi level specification...	An MLM file: levels, clabjects, mediators (Part III)
Open CatUSE specification...	A CatUSE/CatMLM file (Part IV)

Opening a file with the wrong action produces a parse error, not a silent misinterpretation – if in doubt, the file’s own leading keyword (`model`, `multi_model`, `MLM`, or `category` / `class` at the top level) tells you which one it is.

3.7 Loading a Specification from the Command Line

The command line has no separate “which grammar” switch – `spec_file` is compiled with the entry point matching what a `-nogui` batch run needs, inferred the same way as the table above from the file’s own syntax. A typical scripted check:

```
java -jar use-gui.jar -nogui -q myModel.use myChecks.cmd
```

`myChecks.cmd` runs on startup against the freshly loaded model; the process exits 1 if any constraint failed, 0 otherwise – suitable for a CI step that should fail the build on a broken model.

3.8 A Quick Sanity Check

To confirm the build actually works end to end, try opening one of the worked examples that ship in the repository’s own `examples/` directory, e.g. `examples/USE/202608282106-Vehicle.use`, with “Open specification...”. Chapter 4 walks through a small model like this one in full, from writing it to checking its first invariant.

Chapter 4

A Quickstart Tutorial

This chapter is a placeholder.

Part II

Modeling in Plain USE

Chapter 5

Class Diagrams

This chapter is a placeholder.

Chapter 6

OCL Constraints

This chapter is a placeholder.

Chapter 7

The OCL Standard Library

This chapter is a placeholder.

Chapter 8

System States and the Shell

This chapter is a placeholder.

Chapter 9

Validating Specifications

This chapter is a placeholder.

Chapter 10

The Graphical User Interface

This chapter is a placeholder.

Chapter 11

Model Unit Tests

This chapter is a placeholder.

Part III

Multi-Level Modeling with MLM-USE

Chapter 12

Concepts

This chapter is a placeholder.

Chapter 13

MLM-USE Syntax in Detail

This chapter is a placeholder.

Chapter 14

Multi-Level Shell Commands

This chapter is a placeholder.

Chapter 15

Worked Examples

This chapter is a placeholder.

Chapter 16

Validating Multi-Level Models

This chapter is an early draft: the underlying design question — how **should** a multi-level model be validated, and how much of that can reasonably be retrofitted onto tools that were never built with levels in mind — is still open. What follows is accurate as of this writing, but expect it to be revised as that design settles.

Chapter 9 covers validation for plain USE. Everything there — structural checking, invariant evaluation, snapshot generation, bounded model finding — exists as a real, usable tool. This chapter is about what happens to each of those tools once levels, clabjects, and inter-level constructs enter the picture.

16.1 What “Validating a Model” Means in USE

USE actually ships several distinct things under the umbrella of “validation,” and it matters which one is meant:

- **Structural checking** — does the model itself parse and type-check (multiplicities, navigability, redefinition rules, and so on)? This is what the compiler already does; nothing here is specific to runtime state at all.
- **Invariant evaluation against a system state** — given a concrete, already-populated set of objects and links, does every invariant hold? This is ordinary OCL evaluation, nothing more.
- **Snapshot generation** (ASSL, the `generate` shell commands) — backtracking search that actually **builds** a concrete population of objects and links satisfying a set of constraints, guided by a user-written procedure (`Create`, `CreateN`, `Insert`, `Try`, `Any`, ...). This is the closest thing USE has to “show me a model of my constraints.”
- **Bounded model finding** — a SAT/CSP-based search over a finite-size universe, bundled with this fork as a plugin (see 17.4 in the next chapter). Conceptually similar to Alloy’s Kodkod-based analyzer.

Note

`checkPartialSatisfiability()` / `checkWellDefinednessState()` — the two methods behind well-definedness checking — are **not** a solver. Each call constructs exactly one, arbitrary, concrete witness state (every attribute defaulted to `UndefinedValue`) and evaluates the invariant against **that single state**. It is not an existential search over all possible states, and a **false** result does not mean “no witness exists” — only that this particular default-constructed one doesn’t satisfy it. It is easy to over-read what this check actually proves; it proves less than “satisfiable” or “unsatisfiable” in the logical sense.

16.2 The Multi-Level Blind Spot

None of the four tools above were designed with multi-level constructs in mind, and this is not a minor omission:

- A clabject’s **cancellation** (a removed attribute, a renamed attribute, a removed role, a removed constraint — see Chapter 12) is resolved on demand, in memory, by `MInternalClassImpl`’s own bookkeeping. A plain-USE-shaped tool that simply walks `MClass` `!=` `MAssociation` navigation — which is exactly what `MSystem` and the ASSL generator (`GGenerator`) do — sees the **raw**, unflattened per-level class, cancellation and all still present.
- **Inter-classes, inter-associations, and inter-constraints** — constructs owned by no single constituent model at all — live only in the multi-level model’s own bookkeeping (`MMultiModel.interClasses()` `!=` `interAssociations()` `!=` `interConstraints()`). A tool scoped to one constituent `MModel` at a time never sees them.

Handed an `MMultiLevelModel` directly, `GGenerator` happily starts generating against the **raw** per-level classes — not an error, not a warning, just silently wrong: cancelled attributes reappear, renamed ones keep their original name, and inter-level constraints are never checked at all because the generator never even looks at them. This was confirmed directly against this fork’s own `GGenerator` and `MSystem` code, not assumed.

The bundled bounded-model-finder plugin appears, from its bytecode, to have the identical scoping (per-`MModel`, not multi-level-aware) — see 17.4 for the caveats on that finding, since it was established by inspection rather than by actually running it against a multi-level model.

16.3 Two Different Ways to Think About the Problem

There are two genuinely different research angles on “validate a model with an unbounded, or merely very large, population of linked objects,” worth keeping apart:

1. **Bounded, concrete techniques.** Snapshot generation and bounded model finding both work by actually constructing — or searching over — specific, finite object populations. They tell you something true about the instances they build, but they say nothing about instance counts beyond whatever bound was configured. This is what USE actually does today, multi-level or not.
2. **Abstract, unbounded reasoning.** Shape analysis, and in particular parametric shape analysis via three-valued logic (TVLA) [2], reasons about the shape of a linked data structure — how many objects, of what class, connected how — **without ever instantiating a specific population at all**. This is the right theoretical framing for a question like “can this multi-level model’s constraints ever be satisfied by **any** size of population,” but nothing in this fork attempts it; it is mentioned here as a pointer for future work, not as a described feature.

A related, purely theoretical point worth knowing when judging what’s achievable here at all: OCL’s own expressive power depends on which version is in play. OCL 1.4/1.5 expressions are Turing-complete; OCL 2.0 expressions, restricted to guarantee termination, are exactly the primitive recursive functions [1]. This fork was confirmed (empirically, not merely by reading the grammar) to behave like the **unrestricted**, Turing-complete variant — unbounded recursion runs until a genuine `StackOverflowError` (caught and re-thrown as a plain `RuntimeException`, not statically rejected). That matters here because it rules out any hope of a general, exhaustive, **static** verification procedure for arbitrary OCL constraints in this system, multi-level or not: the underlying constraint language is, by construction, too expressive for that. Whatever validation

strategy this fork eventually settles on for multi-level models, it will — like every other OCL tool of this kind — be bounded, or heuristic, or both, not a decision procedure.

Warning

This is an open design area, not a finished feature. The next chapter describes the one concrete step taken so far — making the existing bounded tools (specifically, ASSL) usable against a multi-level model at all, by flattening it first — and is explicit about what still isn't wired up.

Chapter 17

The Flattening Bridge

This chapter describes the one concrete step taken so far on the open question raised in Chapter 16: making an existing bounded validation tool — specifically, ASSL snapshot generation — actually work correctly against a multi-level model, rather than silently generating against its raw, unflattened structure. It does not resolve the larger design question; it makes one tool usable while that question stays open, and it is explicit below about what still isn't done.

17.1 The Idea

The fix isn't to make `GGenerator` multi-level-aware — that would be a substantially larger undertaking than this fork's scope. It's to hand it something it already understands correctly: an ordinary, single-level `MModel`.

Chapter 28 describes `FlatUseTextRenderer` — the tool that computes exactly this. Given a compiled `MMultiLevelModel`, it renders every class's fully resolved, post-cancellation structure (attributes, roles, invariants — cancellations already applied) as ordinary plain-USE text, with every inter-class, inter-association, and inter-constraint folded in and no multi-level syntax left anywhere in the output. Recompiling that text with the ordinary `USECompiler` (not `USECompilerMLM`) yields a plain `MModel` that `GGenerator` was always designed for.

The pipeline, end to end:

```
compiled MMultiLevelModel
    | FlatUseTextRenderer.render(...)
    v
flattened plain-USE text (a "model ... end" block, no MLM syntax)
    | USECompiler.compileSpecification(...)
    v
plain MModel
    | new MSystem(...)
    v
MSystem --- system.generator().startProcedure(...) now just works
```

17.2 FlatUseSystemFactory

`org.tzi.use.tools.catuselatex.FlatUseSystemFactory` packages the last three steps of that pipeline as a single call:

```
MSystem system = FlatUseSystemFactory.createFlattenedSystem(
    mlm,                // an already-compiled MMultiLevelModel
    "MyModelName",     // display name for the flattened "model" block
    err                 // PrintWriter for a recompilation failure, if any
);
```

```
if (system != null) {
    system.generator().startProcedure(callString, args);
}
```

A `null` return means the flattening itself produced text that failed to recompile — a bug in `FlatUseTextRenderer`, not a problem with the input model (which already compiled successfully as a multi-level model to get this far at all).

The returned `MSystem` is a **fresh, independent** system over the flattened model — a different object from any `MSystem` the caller may already have wrapping the original `mlm`. Generation results (objects, links, invariant checks) live in the returned system, not in whichever system `mlm` itself came from. A caller juggling both needs to keep that distinction straight, exactly as with any two unrelated `MSystem` instances.

17.3 What’s Verified

- Every worked example under `examples/MLM-USE/` and `examples/CatMLM/` — including a 1000-plus-class stress-test model — flattens and recompiles cleanly as plain USE.
- `FlatUseSystemFactoryTest` runs a real ASSL procedure end to end against a flattened multi-level model via `system.generator().startProcedure(...)`, and confirms the objects it creates actually land in the resulting system state. Nothing in this codebase exercised `startProcedure` against any model at all before this test existed.

17.4 The Bundled Model-Finder Plugin

This fork ships a second bounded validation tool alongside ASSL: `use-assembly/src/main/resources/plugins` a real, bundled SAT/CSP-based model validator (built on `sat4j` and `choco-solver`, conceptually similar to Alloy’s Kodkod-based analyzer), loaded into `use-gui` at startup via its `PluginRegistry`. It is a pre-built binary jar shipped as a packaging resource, not a Maven dependency of any module’s own source — worth knowing, since a plain source or POM grep for a solver dependency will not find it.

From its bytecode, the plugin appears to be scoped the same way `GGenerator` is: per constituent `MModel`, with no visible awareness of clabjects, cancellation, or inter-level constructs. That finding rests on inspection, not execution — unlike the ASSL generator (see 17.3), this plugin has not actually been run against a multi-level model, flattened or not, so its blind spot is inferred rather than empirically confirmed. Whether the flattening bridge above applies to it as directly as it did to ASSL — the plugin has its own GUI-integrated invocation path, not a bare `MSystem` call — is itself an open question.

17.5 What’s Not Yet Wired Up

Warning

Two things deliberately stop here, both because they raise design questions of their own rather than being simple wiring:

- **The interactive Shell.** `cmdGenStartProcedure` (the shell command behind `gen startprocedure`) still operates directly on `fSession.system()` — whatever the user last `open`’d, multi-level or not — with no flattening step at all. Swapping in a flattened system just for that one call raises an immediate follow-on question: which system do the **later** generation commands (`gen result`, `gen invariant`,

and friends) then operate on? The result lives in the flattened system, not the original one, so every one of those commands would need to know to keep using the flattened system instead of silently looking at the wrong (and un-generated-into) original. That's a real piece of session-state design, not yet attempted.

- **The bundled bounded model finder plugin** (see [17.4](#)) has not been run through this same bridge at all. Its blind spot was established by bytecode inspection, not by actually exercising it against a multi-level model the way ASSL now has been.

Part IV

CatUSE and CatMLM

Chapter 18

Motivation

This chapter is a placeholder.

Chapter 19

CatMLM Syntax in Detail

This chapter is a placeholder.

Chapter 20

Known Expressiveness Gaps

This chapter documents constructs that are deliberately unsupported, or supported only in a restricted form, in MLM-USE and CatMLM – as opposed to bugs, which belong in the issue tracker, not here. Each entry says what’s restricted, why, and what to write instead. The full empirical investigation behind each entry (reproducers, root causes in the Java source, considered alternatives) lives in [known-issues.org](https://github.com/known-issues.org) at the repo root; this chapter gives the user-facing summary.

20.1 Association classes are per-model, not multi-level

20.1.1 The restriction

An `associationclass` declared directly in `inter-classes` is a full citizen of a multi-level model. The same declaration inside an ordinary `model ... end` block is rejected at compile time with a clear error naming the class and model.

20.1.2 Why

An association class is simultaneously a class and an association. Every other multi-level construct keeps those two facets on entirely separate tracks – a `clabject` edge refines a class one level down, an `assoclink` edge refines an association one level down, and nothing in the framework lets one edge do both to the same declaration at once. Giving a per-model association class real multi-level behavior would mean deciding what it means for a single declaration to be refined as a class **and** as an association simultaneously, including how the two refinements stay consistent with each other – semantics that don’t exist yet in the theory this tool implements. That is a research question, not an implementation gap, so it is intentionally out of scope.

20.1.3 What to write instead

- If the association class is inherently a bridge between two different models (or an inter-class and a model class), declare it directly in `inter-classes` – this already works. See `examples/MLM-USE/202608282106-InterAssociationClassBridge.use`.
- If it’s local to one model, reify the edge as an ordinary class plus two ordinary associations, one to each endpoint. This gives up nothing practical: the reified edge is still an object, so population-bound invariants (`allInstances()->size()`) apply to it exactly as they would to a real association class. (`examples/` carries no worked file for this pattern specifically – it’s reserved for constructs that work without qualification, and this one exists only because of the restriction above.)

20.2 CatMLM’s `instanceOf` only ever targets a level’s declared parent

20.2.1 The restriction

A CatMLM level is declared as `model ID1 < ID2`, and every clabject in that level instantiates a powerclass from exactly one level: `ID2`, the one named in that header. There is no syntax for a clabject to name a powerclass from any other level.

20.2.2 Why this isn’t a gap in practice

Because `instanceOf` is always resolved against exactly the declared parent’s class table, reusing a class name at another level – even at every level of a chain – is never ambiguous: there is only ever one table consulted, so a same-named class anywhere else is simply never looked at. (Same-level subclassing, `<`, has the identical property for the unrelated reason that it’s resolved purely within the declaring level’s own table.) This is locked in as a regression test, `CatMLMLevelNameDuplicationTest`, which reuses one class name at every level of a three-level chain – including a same-level subclass sharing the name too – and checks every instantiation still resolves to exactly its own declared parent.

Note that this is a restriction of CatMLM’s front-end syntax specifically, not of MLM-USE’s underlying multi-level model: a plain MLM-USE mediator names its parent model by name (`mediator Child < Parent`), with no adjacency requirement at all, so a lower-level class instantiating a class from a non-adjacent ancestor level is already expressible in plain MLM-USE today. CatMLM simply has no surface syntax to ask for that yet.

20.2.3 What to write instead

If a class genuinely needs to instantiate a powerclass from further up than its immediate CatMLM parent, drop to plain MLM-USE for that level and name the target model explicitly in the `mediator ... < ...` declaration.

20.3 Other CatMLM gaps (unchanged from earlier passes)

CatMLM’s `catUSE_model` has no `operations` section, no `inter-*` sections (constraints/associations/classes), and no per-clabject attribute rename – cancellation is only via `catAtt/=catConstr` tags on the powerclass’s own declaration, applying to every instantiator, not selectively. Drop to plain MLM-USE for any of these.

Chapter 21

Worked Examples

This chapter is a placeholder.

Chapter 22

The CatUSE-to-L^AT_EX Study Tool

This chapter is a placeholder.

Part V

Extending the Implementation

Chapter 23

Codebase Tour

This chapter is a placeholder.

Chapter 24

The Grammar and Parser

This chapter is a placeholder.

Chapter 25

AST and Compilation Pipeline

This chapter is a placeholder.

Chapter 26

The Core M-Model

This chapter is a placeholder.

Chapter 27

The Multi-Model and Multi-Level-Model Classes

This chapter is a placeholder.

Chapter 28

Flattening and Rendering

This chapter is a placeholder.

Chapter 29

Testing the Compiler

This chapter is a placeholder.

Chapter 30

Releasing a New Version and Writing Plugins

This chapter is a placeholder.

Part VI

History

Chapter 31

History of USE

This chapter is a placeholder.

Chapter 32

History of the MLM-USE/CatUSE Fork

This chapter is a placeholder.

Part VII

Appendices

Appendix A

The USE Grammar

The base USE language grammar (classes, associations, OCL constraints, SOIL statements), with no multi-level-modeling and no CatMLM extensions. Reproduced verbatim from `USE.ebnf` at the repository root, which is generated/maintained directly from the actual grammar sources (`USEBase.gpart`, `OCLBase.gpart`, `SoilBase.gpart`, `OCLLexerRules.gpart`) rather than transcribed by hand, so this appendix can never drift from the real, compiled grammar.

```
=====
USE Language Grammar -- Context-Free Grammar (EBNF)
=====
```

```
This is the BASE layer: the classic, single-level USE language (classes
,
associations, OCL constraints, SOIL statements) as it stands in this
fork
today -- with NO multi-level-modeling and NO CatMLM extensions. It is
one
of three layered EBNF files describing this fork's composed grammar:
```

```
USE.ebnf          (this file)  -- the base language, standalone
MLM-USE.ebnf      -- the DELTA that BGU's multi-level-
    modeling
                                extension (MLM, clabject, mediator,
                                assoclink) adds on top of this file
CatMLM.ebnf       -- the DELTA that the CatMLM/"CatUSE"
    front-
                                end syntax sugar adds on top of MLM
                                -USE
```

```
Each delta file only lists rules that are new or changed at that layer;
anything not mentioned there is reused unchanged from the layer(s)
below.
```

```
Source: derived from use-core/src/main/resources/grammars/base/USEBase.
gpart
+ OCLBase.gpart + SoilBase.gpart + OCLLexerRules.gpart, with the
multi-level-modeling rules (multi_level_model, mediator, clabject,
assoclink, and the "roles" contextual keyword) excluded -- those are
documented in MLM-USE.ebnf instead. See that file's header for why they
live in this same physical .gpart file despite being a separate layer
conceptually.
```

```
This file is a *pure* CFG: all embedded Java semantic actions, AST-
```

```

builder
calls, @init blocks and ANTLR "returns [...]" clauses have been
stripped
out, leaving only the production rules. Notation:

::=      "is defined as"
|        alternation
[ x ]    x is optional
{ x }    x may repeat zero or more times
{ x }+   x may repeat one or more times
'x'      literal terminal text
UPPERCASE reference to a lexical token (defined in the LEXICAL
GRAMMAR
        section at the end)
(* ... *) comment

```

Entry points (which rule the parser starts from depends on what kind of spec is being parsed):

```

model          -- a single classic USE class-diagram
  specification
multi_model    -- a file containing several 'model' blocks
  plus
                inter-model associations/constraints (this
                is
                a base-language feature: no clabjects, no
                mediators, no levels -- just several
                ordinary
                models glued by inter-* declarations)
expressionOnly -- parsing a bare OCL expression (e.g. in the
  shell)
typeOnly       -- parsing a bare type
statOnly       -- parsing a bare SOIL statement

```

Note: 'multi_level_model' (the "MLM ..." entry point) is NOT part of this file -- it's MLM-USE.ebnf's entry point, defined on top of multi_model_core below (see that file).

SECTION 1 -- CORE MODEL GRAMMAR

```

(* ---- Multi-model (several ordinary models + inter-model glue) ----
*)

multi_model ::= multi_model_core EOF

multi_model_core ::= [ 'multi_model' IDENT ]
                    internal_model { internal_model }
                    [ 'inter-enums'      { enumTypeDefinition } ]
                    [ 'inter-classes'     {
                      interGeneralClassifierDefinition } ]
                    [ 'inter-associations' {
                      interAssociationDefinition } ]
                    [ 'inter-constraints'  { interInvariant |

```

```

interPrePost } ]

(* multi_level_model (MLM-USE.ebnf) wraps this same rule, unchanged,
   with
   a leading "MLM" IDENT and a trailing sequence of mediators. *)

interGeneralClassifierDefinition ::= annotationSet [ 'abstract' ]
    ( classDefinition
    | interAssociationClassDefinition
    | signalDefinition )

interAssociationClassDefinition ::= ('associationClass' | '
associationclass') IDENT
    [ '<' idList ]
    [ 'between' interAssociationEnd { interAssociationEnd }+ ]
    [ 'attributes' { attributeDefinition } ]
    [ 'operations' { operationDefinition } ]
    [ 'constraints' { invariantClause } ]
    [ 'statemachines' { stateMachine } ]
    [ 'aggregation' | 'composition' ]
    'end'

interAssociationDefinition ::= annotationSet
    ('association' | 'aggregation' | 'composition') IDENT
    'between' interAssociationEnd { interAssociationEnd }+
    'end'

interAssociationEnd ::= annotationSet
    ( (IDENT '@' IDENT) | IDENT ) '[' multiplicity ']'
    [ 'role' IDENT ]
    { 'ordered'
    | 'subsets' IDENT
    | 'union'
    | 'redefines' IDENT
    | ('derived' | 'derive') [ '(' elemVarsDeclaration ')' ] '='
      expression
    | 'qualifier' paramList
    }
    [ ';' ]

interInvariant ::= 'context'
    [ IDENT { ',' IDENT } ':' ]
    (multiType | simpleType)
    { invariantClause }

interPrePost ::= 'context'
    ( (IDENT '::' IDENT paramList [ ':' type ])
    | (multiType '::' IDENT paramList [ ':' type ]) )
    { prePostClause }+

(* ---- Single classic USE model ---- *)

model ::= internal_model EOF

internal_model ::= annotationSet 'model' IDENT
    { generalClassifierDefinition
    | associationDefinition
    | ( 'constraints' { invariant | prePost } )

```

```

    | enumTypeDefinition
  }

enumTypeDefinition ::= annotationSet 'enum' IDENT '{' idList '}' [ ';' ]

dataTypeDefinition ::= 'dataType' IDENT
  [ '<' idList ]
  [ 'operations' { operationDefinition } ]
  [ 'constraints' { invariantClause } ]
  'end'

generalClassifierDefinition ::= annotationSet [ 'abstract' ]
  ( classDefinition
  | dataTypeDefinition
  | associationClassDefinition
  | signalDefinition )

classDefinition ::= 'class' IDENT
  [ '<' idList ]
  [ 'attributes' { attributeDefinition } ]
  [ 'operations' { operationDefinition } ]
  [ 'constraints' { invariantClause } ]
  [ 'statemachines' { stateMachine } ]
  'end'

associationClassDefinition ::= ('associationClass' | 'associationclass
  ') IDENT
  [ '<' idList ]
  [ 'between' associationEnd { associationEnd }+ ]
  [ 'attributes' { attributeDefinition } ]
  [ 'operations' { operationDefinition } ]
  [ 'constraints' { invariantClause } ]
  [ 'statemachines' { stateMachine } ]
  [ 'aggregation' | 'composition' ]
  'end'

attributeDefinition ::= annotationSet IDENT ':' type
  [ ( (('derive'|'derived') (':'|'=') expression)
    | ('init' (':'|'=') expression) ) ]
  [ ';' ]

operationDefinition ::= annotationSet IDENT paramList superParamList
  [ ':' type ]
  [ ('=' expression) | blockStat ]
  { prePostClause }
  [ ';' ]

(* superParamList: an optional second, plain parenthesized identifier
   list
   right after an operation's normal parameter list, used to mark which
   parameters an operation named after its own classifier (a
   constructor)
   forwards to a super-classifier rather than storing as its own
   attribute.
   Independent of multi-level modeling -- applies to any classifier. *)
superParamList ::= [ '(' [ IDENT { ',' IDENT } ] ')' ]

```



```

associationDefinition ::= annotationSet
    ('association' | 'aggregation' | 'composition') IDENT
    'between' associationEnd { associationEnd }+
    'end'

associationEnd ::= annotationSet IDENT '[' multiplicity ']'
    [ 'role' IDENT ]
    { 'ordered'
    | 'subsets' IDENT
    | 'union'
    | 'redefines' IDENT
    | ('derived' | 'derive') [ '(' elemVarsDeclaration ')' ] '='
      expression
    | 'qualifier' paramList
    }
    [ ';' ]

multiplicity      ::= multiplicityRange { ',' multiplicityRange }
multiplicityRange ::= multiplicitySpec [ '..' multiplicitySpec ]
multiplicitySpec  ::= INT | '*'

invariant ::= 'context'
    [ IDENT { ',' IDENT } ':' ]
    simpleType
    { invariantClause }

invariantClause ::= annotationSet 'inv' [ IDENT ] ':' expression
    | 'existential' 'inv' [ IDENT ] ':' expression

prePost ::= 'context' IDENT '::' IDENT paramList [ ':' type ]
    { prePostClause }+

prePostClause ::= annotationSet ('pre' | 'post') [ IDENT ] ':'
    expression

annotationSet    ::= { annotation }
annotation        ::= '@' IDENT '(' annotationValues ')'
annotationValues  ::= [ annotationValue ] { ',' annotationValue }
annotationValue   ::= IDENT '=' NON_OCL_STRING

stateMachine ::= 'psm' IDENT
    'states'      { stateDefinition }+
    'transitions' { transitionDefinition }+
    'end'

stateDefinition ::= IDENT [ ':' IDENT ] [ '[' expression ']' ]

transitionDefinition ::= IDENT '->' IDENT
    [ '{' [ '[' expression ']' ]
      ( event | IDENT '(' [ paramList ] ')' )
      [ '[' expression ']' ]
    '}' ]

event ::= 'create'

signalDefinition ::= 'signal' IDENT
    [ '<' idList ]
    [ 'attributes' { attributeDefinition } ]

```

```

    [ 'constraints' { invariantClause } ]
    'end'

(* ---- Contextual ("soft") keywords ----
The following are NOT reserved words at the lexer level: each is an
ordinary IDENT token that is only treated as a keyword when a
semantic
predicate confirms its exact spelling at that grammar position. This
lets 'union', 'role', 'association', 'aggregation', 'composition',
'dataType', 'class', 'signal', 'derived', 'derive', 'init', and
'qualifier' remain usable as ordinary identifiers elsewhere (e.g. as
a
class or attribute name) without being reserved everywhere. MLM-USE.
ebnf
and CatMLM.ebnf each add their own such keywords, scoped to their
own
grammar positions only -- see those files. *)

keyUnion      ::= IDENT      (* text must equal "union" *)
keyAssociation ::= IDENT      (* text must equal "association" *)
keyRole       ::= IDENT      (* text must equal "role" *)
keyComposition ::= IDENT      (* text must equal "composition" *)
keyAggregation ::= IDENT      (* text must equal "aggregation" *)
keyDataType   ::= IDENT      (* text must equal "dataType" *)
keyClass      ::= IDENT      (* text must equal "class" *)
keySignal     ::= IDENT      (* text must equal "signal" *)
keyDerived    ::= IDENT      (* text must equal "derived" *)
keyDerive     ::= IDENT      (* text must equal "derive" *)
keyInit       ::= IDENT      (* text must equal "init" *)
keyQualifier  ::= IDENT      (* text must equal "qualifier" *)

=====

SECTION 2 -- OCL EXPRESSION GRAMMAR
=====

expressionOnly ::= expression EOF

expression ::= { 'let' IDENT [ ':' type ] '=' expression
                { ',' IDENT [ ':' type ] '=' expression }
                'in' }
            conditionalImpliesExpression

paramList ::= '(' [ variableDeclaration { ',' variableDeclaration } ]
            ')',

idList ::= IDENT { ',' IDENT }

variableDeclaration ::= IDENT ':' type

conditionalImpliesExpression ::= conditionalOrExpression { 'implies'
    conditionalOrExpression }
conditionalOrExpression      ::= conditionalXOrExpression { 'or'
    conditionalXOrExpression }
conditionalXOrExpression     ::= conditionalAndExpression { 'xor'
    conditionalAndExpression }
conditionalAndExpression     ::= equalityExpression { 'and'

```

```

equalityExpression }
equalityExpression      ::= relationalExpression      { ('=' |
  '<>') relationalExpression }
relationalExpression    ::= additiveExpression      { ('<' |
  '>' | '<=' | '>=') additiveExpression }
additiveExpression      ::= multiplicativeExpression { ('+' |
  '-') multiplicativeExpression }
multiplicativeExpression ::= unaryExpression        { ('*' |
  '/' | 'div') unaryExpression }

unaryExpression ::= ('not' | '-' | '+') unaryExpression
                | postfixExpression

postfixExpression ::= primaryExpression { ('.' | '->') propertyCall }

primaryExpression ::= literal
                  | objectReference
                  | propertyCall
                  | '(' expression ')'
                  | ifExpression
                  | IDENT '.' 'allInstances' [ '@' 'pre' ] [ '(' ')' ]
                  | IDENT '@' IDENT '.' 'allInstances' [ '@' 'pre' ]
                    [ '(' ')' ]
                  | IDENT '.' 'byUseId' '(' expression ')' [ '@' '
                    pre' ]

objectReference ::= '@' IDENT

(* propertyCall dispatches on the identifier + following token:
   - a recognized "query" name immediately followed by '(' ->
     queryExpression
   - 'iterate' ->
     iterateExpression
   - 'oclIsInState' / 'oclInState' ->
     inStateExpression
   - 'oclAsType' / 'oclIsKindOf' / 'oclIsTypeOf'
     / 'selectByType' / 'selectByKind' ->
     typeExpression
   - anything else ->
     operationExpression
   propertyCall carries the preceding expression (the navigation source
     , or
     null at the start of an expression) and whether it was reached via
     '->'. *)

propertyCall ::= queryExpression
              | iterateExpression
              | operationExpression
              | typeExpression
              | inStateExpression

queryExpression ::= IDENT '(' [ elemVarsDeclaration '|' ] expression ')'
,

(* IDENT here must be one of the recognized query operator
   names,
   e.g. select, reject, collect, exists, forAll, isUnique,
   sortBy,

```

```

    one, any, collectNested, ... *)

iterateExpression ::= 'iterate' '('
    elemVarsDeclaration ';'
    variableInitialization '|'
    expression
    ')'

operationExpression ::= IDENT
    [ '[' expression { ',' expression } ']'
    [ '[' expression { ',' expression } ']' ] ]
    [ '@' 'pre' ]
    [ '(' [ expression { ',' expression } ] ')' ]

inStateExpression ::= ('oclIsInState' | 'oclInState') '(' IDENT ')'

typeExpression ::= ('oclAsType' | 'oclIsKindOf' | 'oclIsTypeOf'
    | 'selectByType' | 'selectByKind') '(' type ')'

elemVarsDeclaration ::= IDENT [ ':' type ] { ',' IDENT [ ':' type ] }

variableInitialization ::= IDENT ':' type '=' expression

ifExpression ::= 'if' expression 'then' expression 'else' expression 'endif'

literal ::= 'true' | 'false'
    | INT | REAL | STRING
    | '#' IDENT
    | IDENT '::' IDENT
    | multiType '::' IDENT
    | collectionLiteral
    | emptyCollectionLiteral
    | undefinedLiteral
    | tupleLiteral
    | '*'
    (* unlimited natural, e.g. in
    multiplicities *)

collectionLiteral ::= ('Set' | 'Sequence' | 'Bag' | 'OrderedSet')
    '{' [ collectionItem { ',' collectionItem } ] '}'

collectionItem ::= expression [ '..' expression ]

emptyCollectionLiteral ::= 'oclEmpty' '(' collectionType ')'
    | collectionType '{' '}'

undefinedLiteral ::= 'oclUndefined' '(' type ')'
    | 'Undefined'
    | 'null' '(' type ')'
    | 'null'

tupleLiteral ::= 'Tuple' '{' tupleItem { ',' tupleItem } '}'

tupleItem ::= IDENT ( ':' type '=' expression ) | ( ':' | '=' )
    expression )

type ::= simpleType | collectionType | tupleType | multiType

```

```

typeOnly ::= type EOF

simpleType ::= IDENT
multiType  ::= IDENT '@' IDENT
              (* multiType (Level@Name) is a base-language token shape reused
                heavily by MLM-USE for fully-qualified names -- see MLM-USE.
                ebnf. *)

collectionType ::= ('Collection' | 'Set' | 'Sequence' | 'Bag' | '
    OrderedSet')
                '(' type ')',

tupleType ::= 'Tuple' '(' tuplePart { ',' tuplePart } ')',

tuplePart ::= IDENT ':' type

=====

SECTION 3 -- SOIL STATEMENT GRAMMAR (object/link manipulation language)
=====

statOnly ::= stat EOF

stat ::= singleStat { ';' singleStat }

singleStat ::= emptyStat
              | statStartingWithExpr
              | varAssignStat
              | objCreateStat
              | objDestroyStat
              | lnkInsStat
              | lnkDelStat
              | condExStat
              | iterStat
              | whileStat
              | blockStat

emptyStat ::= (* empty *)

(* An expression-statement, optionally followed by an attribute
   assignment on that expression (i.e. distinguishes a bare operation
   call from "expr.attr := rvalue"). *)
statStartingWithExpr ::= inSoilExpression [ attAssignStat ]

varAssignStat ::= IDENT ':=' rValue

attAssignStat ::= '.' IDENT ':=' rValue

objCreateStat ::= 'new' simpleType
                [ '(' [ inSoilExpression ] ')' ]
                [ 'between' '(' rValListMin2WithOptionalQualifiers ')' ]

objDestroyStat ::= 'destroy' exprListMin1

lnkInsStat ::= 'insert' '(' rValListMin2WithOptionalQualifiers ')' '
    into'
                ( IDENT | IDENT '@' IDENT )

```

```

rValListMin2WithOptionalQualifiers ::=
    rValue [ '{' rValList '}' ]
    ',' rValue [ '{' rValList '}' ]
    { ',' rValue [ '{' rValList '}' ] }

lnkDelStat ::= 'delete' '(' rValListMin2WithOptionalQualifiers ')' ' '
    from' IDENT

condExStat ::= 'if' inSoilExpression 'then' statOrImplicitBlock
    [ 'else' statOrImplicitBlock ]
    'end'

iterStat ::= 'for' IDENT 'in' inSoilExpression 'do' statOrImplicitBlock
    'end'

whileStat ::= 'while' inSoilExpression 'do' statOrImplicitBlock 'end'

blockStat ::= 'begin'
    [ 'declare' variableDeclaration { ',' variableDeclaration } ';'
    ]
    stat
    'end'

implicitBlockStat ::= 'declare' variableDeclaration { ','
    variableDeclaration } ';' stat

statOrImplicitBlock ::= stat | implicitBlockStat

rValue ::= inSoilExpression | objCreateStat

rValList      ::= [ rValListMin1 ]
rValListMin1 ::= rValue { ',' rValue }
rValListMin2 ::= rValue ',' rValue { ',' rValue }

inSoilExpression ::= expression

exprList      ::= [ exprListMin1 ]
exprListMin1 ::= inSoilExpression { ',' inSoilExpression }
exprListMin2 ::= inSoilExpression ',' inSoilExpression { ','
    inSoilExpression }

identList      ::= [ identListMin1 ]
identListMin1 ::= IDENT { ',' IDENT }

=====

SECTION 4 -- LEXICAL GRAMMAR (tokens)
=====

Shared verbatim by all three layers (USE, MLM-USE, CatMLM) -- none of
them
add or change a token definition.

(* Whitespace / comments -- discarded (sent to the HIDDEN channel) *)
WS      ::= ' ' | '\t' | '\f' | NEWLINE
SL_COMMENT ::= ('//' | '--') { any char except NEWLINE }
ML_COMMENT ::= '/*' { any char, non-greedy } '*/'

```

```

NEWLINE      ::= '\r\n' | '\r' | '\n'

(* Punctuation / operator tokens *)
ARROW        ::= '->'
AT           ::= '@'
BAR          ::= '|'
COLON        ::= ':'
COLON_COLON  ::= '::'
COLON_EQUAL  ::= ':='
COMMA        ::= ','
DOT          ::= '.'
DOTDOT       ::= '..'
EQUAL        ::= '='
GREATER      ::= '>'
GREATER_EQUAL ::= '>='
HASH         ::= '#'
LBRACE       ::= '{'
LBRACK       ::= '['
LESS         ::= '<'
LESS_EQUAL   ::= '<='
LPAREN       ::= '('
MINUS        ::= '-'
NOT_EQUAL    ::= '<>'
PLUS         ::= '+'
RBRACE       ::= '}'
RBRACK       ::= ']'
RPAREN       ::= ')'
SEMI         ::= ';'
SLASH        ::= '/'
STAR         ::= '*'

(* Numbers *)
INT          ::= digit { digit }
              (* digit = '0'..'9' *)
REAL         ::= INT '.' INT [ ('e'|'E') ['+'|'-'] INT ]
              | INT ('e'|'E') ['+'|'-'] INT
RANGE_OR_INT ::= INT (retyped to INT if followed by '..', else to
REAL if it forms a REAL, else stays INT)

(* Strings *)
STRING       ::= '"' { any char except '"' or '\' | ESC } '"'
              (* OCL string literal *)
NON_OCL_STRING ::= "'" { any char except "'" or '\' | ESC } "'"
              (* used only in annotation values *)
ESC          ::= '\' ( 'n'|'r'|'t'|'b'|'f'|'"'|"'"|'\'|
                      | 'u' HEX_DIGIT HEX_DIGIT HEX_DIGIT HEX_DIGIT
                      | '0'..'3' [ '0'..'7' [ '0'..'7' ] ]
                      | '4'..'7' [ '0'..'7' ] )
HEX_DIGIT    ::= '0'..'9' | 'A'..'F' | 'a'..'f'

(* Identifiers *)
IDENT ::= ('$' | letter | '_' ) { letter | '_' | digit }
        (* letter = 'a'..'z' | 'A'..'Z' *)
        (* After matching, ANTLR's literal table reclassifies IDENT
tokens
whose text matches a quoted keyword used elsewhere in the
grammar (e.g. 'class', 'model', 'context', 'inv', 'let',
...)

```

```

as that keyword token instead. The contextual "soft"
keywords
listed above (union, role, association, ...) are NOT handled
this way -- they stay IDENT and are matched only via
semantic
predicate at the point of use. *)

```

NOTES

1. This is a **pure** CFG view of the grammar for reading/reasoning purposes.
The real ANTLR3 grammar (use-core/target/grammars/USE.g, regenerated on every 'mvn compile'/'mvn package' from the .gpart sources) is not equivalent to this file in the formal sense: it interleaves embedded Java actions that build the AST as parsing proceeds, uses a few syntactic-predicate lookaheads (e.g. '(statStartingWithExpr)=>', '(COLON type EQUAL)=>') to resolve local ambiguities that a plain CFG can't express, and relies on semantic predicates for the contextual keywords noted above. Treat this file as documentation, not as something you could feed straight into a parser generator.
2. This file, together with MLM-USE.ebnf and CatMLM.ebnf, reconstructs the full USE.g composition (USEBase.gpart + OCLBase.gpart + SoilBase.gpart + OCLLexerRules.gpart, layered with the multi-level and CatMLM additions)
-- the grammar for ordinary .use / .mlm specification files, not the interactive shell-command grammar (a separate, sixth fragment, ShellCommandBase.gpart, builds that one instead).

Appendix B

The MLM-USE Grammar

The multi-level-modeling extension, as a **delta** on top of the USE grammar in Appendix A: only rules that are new or changed at this layer are listed. Reproduced verbatim from `MLM-USE.ebnf` at the repository root.

This grammar describes syntax only. A construct can parse cleanly here and still be rejected during semantic analysis once it's nested inside a multi-level model – most notably, a `model ... end` block's classifier list reuses the plain USE `associationclass` rule unchanged, so an association class inside a `model` block parses, but is rejected at compile time (it's only accepted in `inter-classes`). See Chapter 20 for this and the other cases where the grammar alone doesn't tell the whole story.

```
=====
MLM-USE Grammar -- DELTA over USE.ebnf (EBNF)
=====
```

```
This file lists ONLY what the BGU multi-level-modeling extension (
  clabjects,
mediators, powerclasses, assoclinks) adds on top of the base language
in
USE.ebnf. Anything not mentioned here -- class/association/constraint/
OCL/
SOIL syntax, the "model"/"multi_model" entry points, etc. -- is exactly
the
grammar in USE.ebnf, unchanged.
```

```
Notation is the same as USE.ebnf:
```

```
 ::=          "is defined as"          |      alternation
[ x ]        x is optional              { x }    x may repeat zero or more
          times
{ x }+       x may repeat one or more times
'x'          literal terminal text
UPPERCASE    reference to a lexical token (see USE.ebnf Section 4)
(* ... *)    comment
```

```
-----
Where this actually lives (important, and the reason the old merged
USE-grammar.ebnf file was misleading about it): despite the project's
directory layout suggesting a clean per-feature split --
```

```
use/USE.gpart          -- (its own header comment claims:) "
multi-
                        level modeling additions (MLM,
```

```

                                clabject,
                                mediator, assoclink) + core UML
                                model
                                syntax"
base/USEBase.gpart            -- "core class/association/constraint
                                syntax"

-- use/USE.gpart in fact contains ONLY grammar boilerplate (package
declarations, lexer error-handling members) and defines NO production
rules at all. Every rule below is physically defined inside
base/USEBase.gpart, interleaved with the base-language rules from
USE.ebnf. Confirmed by diffing USEBase.gpart against its state at
commit
6106270f, the last commit before MLM work began (571a84bb, "Began
implementing MLM, added parsing rules and AST classes") -- multi_level_
model, mediator, clabject, and assoclink are a clean, self-contained
insertion with no further changes anywhere else in the file since. This
file corrects that attribution: everything below is the true MLM-USE
delta, regardless of which physical .gpart file it happens to sit in.
-----

New entry point (alongside 'model' and 'multi_model' from USE.ebnf):
multi_level_model    -- a full multi-level model file (starts with '
MLM')

=====

NEW RULES
=====

multi_level_model ::= 'MLM' IDENT multi_model_core { mediator } EOF
    (* multi_model_core is USE.ebnf's rule, reused completely
    unchanged
    -- an MLM file is exactly "MLM <name>" followed by the same
    several-models-plus-inter-* body multi_model already accepts
    followed by zero or more mediators. "MLM" here is a soft
    keyword
    (keyMLM below), not a global reserved word -- see that
    section. *)

mediator ::= 'mediator' IDENT '<' IDENT
    { clabject | assoclink }
    'end'
    (* "mediator" is a soft keyword (keyMediator below). *)

clabject ::= 'clabject' IDENT ':' IDENT
    [ 'attributes' { (IDENT '->' IDENT) | ('~' IDENT) } ]
    [ 'roles'      { '~' IDENT } ]
    [ 'constraints' { '~' IDENT } ]
    'end'
    (* The child (IDENT before ':') instantiates the powerclass (
    IDENT
    after ':') from the mediator's parent model. The three
    optional
    sections cancel or rename inherited features: attribute

```

```

        renaming
        (IDENT -> IDENT) or removal (~IDENT), role removal, and
        constraint removal -- each by the powerclass's own feature
        name.
        "clabject" is a soft keyword (keyClabject below); "roles" is
        too
        (keyRoles below). *)

assoclink ::= 'assoclink' IDENT ':' IDENT
           (IDENT '->' IDENT)
           (IDENT '->' IDENT)
           'end'

(* An assoclink between two clabjects instantiates an
   association
   between their two powerclasses; the two role-renaming pairs
   bind the child association's two role names to the parent
   association's two role names. "assoclink" is a soft keyword
   (keyAssoclink below). *)

```

NEW CONTEXTUAL ("soft") KEYWORDS

```

keyMLM      ::= IDENT      (* text must equal "MLM"; matched only at the
   start
                           of a multi_level_model *)
keyMediator  ::= IDENT      (* text must equal "mediator"; matched only at
   a
                           mediator block's header *)
keyClabject  ::= IDENT      (* text must equal "clabject"; matched only at
   a
                           clabject declaration's header *)
keyAssoclink ::= IDENT      (* text must equal "assoclink"; matched only
   at an
                           assoclink declaration's header *)
keyRoles     ::= IDENT      (* text must equal "roles"; matched only
   inside the
                           clabject rule's own "roles" section header
                           above *)

```

None of these five existed as soft keywords from the start. All five were originally written as bare ANTLR3 literals directly in these rules, which auto-promotes each one to a keyword reserved *everywhere* in its composed grammar -- not just inside the one multi-level-modeling section it actually belongs to. That silently broke any plain, non-MLM USE model that happened to use "roles", "MLM", "mediator", "clabject", or "assoclink" as an ordinary identifier (e.g. an OCL operation parameter, or an attribute name) -- "roles" was caught first, breaking use-gui's own ShellIT/t109.use integration fixture; the other four were found and fixed as a direct follow-up, confirmed

```

empirically the same way (a plain model with an attribute named "
  clabject"
failed to parse with "mismatched input 'clabject' expecting 'end'"
  before
the fix). "MLM" and "clabject" were bare literals in CatMLM.gpart too,
independently of USEBase.gpart -- since CatMLM is a separately-composed
grammar with its own lexer, both files needed fixing; CatMLM.gpart's
  rules
now reuse the keyMLM/keyClabject rules defined once here, the same way
  they
already reused keyClass and keyAssociation from USE.ebnf. Fixed by
introducing these five soft keywords, the same pattern USE.ebnf's base
contextual keywords already use (keyRole, keyUnion, keyAssociation,
  ...):
confirmed via an isolated git worktree that the "roles" instance of the
  bug
predated any multi-level-modeling work and was purely a base-grammar
  defect
introduced alongside the clabject rule.

```

NOTES

1. Every non-terminal referenced above but not defined here or reused verbatim from USE.ebnf (IDENT, and the multi_model_core it wraps) comes from USE.ebnf. multiType (IDENT '@' IDENT, i.e. "Level@Name") is USE.ebnf's token shape but is where MLM-USE's fully-qualified names live at the semantic level -- the grammar itself doesn't need a separate production for it since ordinary type/simpleType references already parse it.
2. As with USE.ebnf, this is a pure-CFG view for reading/reasoning; the real grammar interleaves Java AST-builder actions (e.g. resolving a mediator's parent model by name, not by declaration order -- a bug found and fixed in this fork's Java layer, not visible at the grammar level at all).
3. See CatMLM.ebnf for the front-end sugar layer built on top of this one -- CatMLM's "category"/"clabject" shorthand and cat* tags desugar, before any of this grammar runs, into exactly the plain MLM-USE AST that parsing the rules above would itself produce.

Appendix C

The CatMLM Grammar

The CatMLM (“CatUSE”) front-end syntax sugar, as a **delta** on top of MLM-USE (Appendix B): CatMLM desugars into exactly the same AST that plain MLM-USE produces. Reproduced verbatim from `CatMLM.ebnf` at the repository root.

```
=====
CatMLM ("CatUSE") Grammar -- DELTA over MLM-USE.ebnf (EBNF)
=====

This file lists ONLY what the CatMLM front-end -- the terser "category
"/
"clabject" surface syntax with cat* tags, informally called "CatUSE" --
  adds
on top of MLM-USE.ebnf. It is a separate, additive grammar: it changes
nothing in USEBase.gpart/USE.ebnf/MLM-USE.ebnf, and everything
  downstream
of parsing (MMultiLevelModel, well-definedness checking, the GUI, ...)
  has
no notion that CatMLM exists at all. A CatMLM parse tree is *desugared
*, in
Java (USECompilerCatUSE.desugar), into the exact same
  ASTMultiLevelModel /
ASTMediator / ASTClabject / ASTModel / ASTClass tree that parsing the
equivalent plain MLM-USE.ebnf source would produce.

Source: use-core/src/main/resources/grammars/catmlm/CatMLM.gpart,
  composed
as CatMLM.gpart + USEBase.gpart + OCLBase.gpart + SoilBase.gpart +
OCLLexerRules.gpart -- a genuinely separate ANTLR grammar/parser
(CatUSEParser) from the one USE.ebnf/MLM-USE.ebnf describe, sharing
  only
the reused-unchanged rules noted below.

Notation is the same as USE.ebnf:
  ::=          "is defined as"          |      alternation
  [ x ]        x is optional             { x }    x may repeat zero or more
              times
  { x }+       x may repeat one or more times
  'x'          literal terminal text
  UPPERCASE    reference to a lexical token (see USE.ebnf Section 4)
  (* ... *)    comment

New entry point:
```

```

catUSE_model    -- a full CatMLM file (also starts with 'MLM' IDENT
--
                see the dispatch note below)

Note on dispatch: catUSE_model and MLM-USE.ebnf's multi_level_model
both
start with the literal sequence 'MLM' IDENT, so a bare file is NOT
self-describing -- there is no sniffing of file content to decide which
grammar to parse it with. The tool chooses: the GUI has a separate "
    Open
CatUSE specification..." action (MainWindow.ActionFileOpenCatUSESpec)
distinct from its plain "Open MLM specification..." action, and
USECompilerCatUSE.compileCatUSESpecification vs. the plain MLM compiler
are two different entry points at the Java level.

=====

NEW RULES
=====

catUSE_model ::= 'MLM' IDENT { catLevel }+ EOF
    (* "MLM" here is MLM-USE.ebnf's keyMLM soft keyword, reused
       unchanged -- not redefined in this file, see below. *)

catLevel ::= 'model' IDENT [ '<' IDENT ]
            { catClassifierDefinition
              | catAssociationDefinition
              | catClabjectDefinition
              | ( 'constraints' { catInvariant } )
            }
    (* Fuses what MLM-USE.ebnf splits across a separate 'model'
       block
       (in multi_model_core) and a separate 'mediator' block: one
       "model ID [< Parent]" section holds both the level's own
       classifiers/associations/constraints AND its clabjects (
           which
       desugar into the *parent* level's mediator). *)

catClassifierDefinition ::= ( 'category' | 'class' ) IDENT
    [ '<' catRefList ]
    [ ':' catRefList ]
    [ 'attributes' { catAttributeDefinition } ]
    [ 'constraints' { catInvariantClause } ]
    'end'
    (* "category X ... end" and "class X ... end" desugar to the
       exact
       same AST class either way -- isCategory is purely a hint for
       a
       human reader (and, incidentally, for the study tool's
           PlantUML
       diagram stereotypes); it is never checked against anything
       and
       decides nothing about the resulting model's semantics.

       The ":" clause is Java-"implements"-style fusion of
           instantiation
       onto the classifier's own header, e.g.:

```

```

        category C2 < D : C3
        attributes
            seats: Integer
        end
    as an alternative to a separate catClabjectDefinition
        statement
    for the same name. It desugars to exactly the same
        ASTCatClabject
    a standalone "clabject C2 : C3 end" would produce, added to
        the
    same level -- so the two spellings are sugar for each other,
        and
    using both for the same class name is not an error: their
    reference lists union (see the NOTES section on silent union
        ). *)

catRefList ::= IDENT
    | '(' IDENT ',' IDENT { ',' IDENT } ') '
    (* One bare identifier, or two-or-more wrapped in parentheses
        --
        used for every class-name reference list in CatMLM: both
        clauses of catClassifierDefinition above, and
        catClabjectDefinition's powerclass list below. The grammar
        itself has no ambiguity that requires the parentheses ":"
        already unambiguously ends a preceding "<" list on its own,
        and a standalone list never sits next to another one) --
        this
        is a readability convention, applied consistently everywhere
        such a list can appear, not a parsing necessity. A single
        name
        is never parenthesized: "(A)" is rejected, only "A" or
        "(A, B, ...)" are valid. *)

catAttributeDefinition ::= IDENT ':' type [ ';' 'catAtt' ] [ ';' ]
    (* Reuses IDENT/COLON/type exactly like USE.ebnf's
        attributeDefinition;
        only the optional trailing "catAtt" tag is new. A catAtt-
        tagged
        attribute is automatically cancelled in every clabject that
        instantiates this (or an inheriting) category. *)

catInvariantClause ::= 'inv' [ IDENT ] ':' [ 'catConstr' ] expression
    (* The "catConstr" tag sits between the colon and the
        expression.
        A catConstr-tagged invariant is automatically cancelled in
        every
        instantiating clabject, the same way catAtt cancels
        attributes. *)

catInvariant ::= 'context'
    [ IDENT { ',' IDENT } ':' ]
    simpleType
    { catInvariantClause }
    (* Mirrors USE.ebnf's invariant rule exactly, just built from
        catInvariantClause instead of invariantClause. *)

catAssociationDefinition ::= ( 'catAssociation' | 'association' | '
    aggregation' | 'composition' ) IDENT

```

```

'between' associationEnd { associationEnd }+
'end'
(* associationEnd is MLM-USE.ebnf/USE.ebnf's rule, reused
   unchanged.
   A catAssociation-tagged association has its far-end role(s)
   cancelled for any clabject instantiating the near end --
       this is
   what stops a clabject instantiating two powerclasses from
       ending
   up with two different roles that happen to share a name. *)

catClabjectDefinition ::= 'clabject'
[ IDENT '<' IDENT ';' ]
IDENT ':' catRefList
'end'
(* One declaration, N powerclasses, and an optional fused same-
   level
   subclass edge, all collapsing to this single rule:
       clabject C : A end
       clabject C : (A, B) end
       clabject E < D; E : B2 end
   The first optional "IDENT < IDENT ;" is a same-level
       superclass
   edge (like MLM-USE.ebnf's own "class X < Y"); the required
   "IDENT : catRefList" lists the one-or-more powerclasses this
   clabject instantiates (parenthesized if more than one -- see
   catRefList above), desugaring to one plain MLM-USE
   'clabject <name> : <powerclass>' per listed powerclass. No
   keyword precedes the powerclass list -- see below. The
       leading
   "clabject" is MLM-USE.ebnf's keyClabject soft keyword,
       reused
   unchanged -- not redefined in this file, see below. *)

```

NEW CONTEXTUAL ("soft") KEYWORDS

```

keyCategory      ::= IDENT    (* text must equal "category"; matched
   only                                     in catClassifierDefinition, choosing
                                           between
                                           "category ID ... end" and "class ID
                                           ... end" *)

keyCatAtt         ::= IDENT    (* text must equal "catAtt" *)
keyCatConstr      ::= IDENT    (* text must equal "catConstr" *)
keyCatAssociation ::= IDENT    (* text must equal "catAssociation" *)

```

(keyMLM and keyClabject, used above, are MLM-USE.ebnf's soft keywords
 --
 CatMLM.gpart reuses those rule definitions rather than declaring its
 own
 "MLM"/"clabject" literals a second time, which would have created the
 same
 "reserved everywhere in this composed grammar's lexer" problem
 independently
 of USEBase.gpart. This mirrors how catClassifierDefinition already

reuses
 USE.ebnf's keyClass, and catAssociationDefinition reuses keyAssociation
 ,
 keyAggregation, and keyComposition.)

Note on "category" and catClabjectDefinition: an earlier version of
 this

grammar also accepted an optional "category" hint immediately after the
 colon in catClabjectDefinition (i.e. "clabject C : category A, B end").
 It was removed: the "<" vs ":" already unambiguously distinguishes
 same-level generalization from cross-level instantiation, so the
 keyword

added no information a reader didn't already have from the symbol
 choice

-- and it was never actually checked against the named class's own
 declaration (a clabject could "instantiate" a plain "class", not just a
 "category", keyword or no keyword, both before and after this change).
 keyCategory today is used exclusively in catClassifierDefinition.

NOTES

1. Every non-terminal referenced above but not defined here -- type,
 expression, simpleType, associationEnd -- is reused completely
 unchanged
 from USE.ebnf/MLM-USE.ebnf. catRefList is CatMLM's own rule (defined
 above): the base grammar's plain idList (a bare comma list, no
 parentheses ever) is no longer used anywhere in CatMLM.gpart at all
 --
 catRefList replaced its two former uses (catClassifierDefinition's
 "<"
 list, catClabjectDefinition's ":" list) and is also used for the new
 ":" clause on catClassifierDefinition.
2. As with the other two files, this is a pure-CFG view for reading and
 reasoning, not something to feed straight into a parser generator:
 the
 real grammar's semantic actions build the desugared AST as parsing
 proceeds (see USECompilerCatUSE.desugar's four translation rules),
 and
 ANTLR's adaptive lookahead resolves a few local ambiguities a plain
 CFG
 can't express -- for instance, a category or clabject literally
 named
 "category" (or "catAtt", "role", etc.) still parses correctly as an
 ordinary identifier wherever an IDENT is expected, backtracking off
 the
 soft-keyword reading when it can't also complete a valid parse.
3. Silent union, and a bug it uncovered: declaring the same class name's
 instantiation or same-level superclassifiers via more than one route
 --
 the fused "":"/<" header on catClassifierDefinition, a separate
 catClabjectDefinition statement, and/or that statement's own fused
 "IDENT < IDENT ;" local-superclass prefix -- is not an error. All of

them contribute to the same union, by design: e.g. a classifier declared with "category C2 < D : C3 end" and, elsewhere in the same level, a separate "clabject C2 : C4 end" both apply to the one class C2, which ends up instantiating both C3 and C4. Building this surfaced a genuine pre-existing bug (not new in this change) in `ASTClassifier.addSuperClassifiers`: it used to **replace** (`fSuperClassifiers = idList`) rather than accumulate, so a class whose same-level superclassifiers were set from two different declaration sites would silently keep only whichever one ran last. Fixed to accumulate instead -- safe for every base-grammar caller too, since none of them ever call it more than once per classifier.

Appendix D

Keyword Reference

This chapter is a placeholder.

Appendix E

Shell Command Reference

This chapter is a placeholder.

Bibliography

- [1] María Victoria Cengarle and Alexander Knapp. OCL 1.4/5 vs. 2.0 expressions: Formal semantics and expressiveness. *Software and Systems Modeling*, 3:9–30, 2004.
- [2] Mooly Sagiv, Thomas Reps, and Reinhard Wilhelm. Parametric shape analysis via 3-valued logic. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 24(3):217–298, 2002.

Index

- CatUSE
 - relationship to MLM-USE, 3
- clabject, 4
- command-line options, 9
- FlatUseSystemFactory, 45
- JDK, 7
- level (multi-level modeling), 4
- Maven, 7
- mediator, 4
- MLM-USE, 3
- opening a specification (GUI), 10
- powerclass, 4
- shell
 - relationship to the GUI, 10
- USE, 3
- validation
 - architectural blind spot, 42
 - bounded model finding, 46
 - flattening bridge, 45
 - multi-level models, 41